PRO-CODE EXTENSION USING ZOHO CATALYST

Aim

To extend the functionality of a low-code application developed using Zoho Creator by integrating Zoho Catalyst for implementing advanced business logic, backend services, and custom application features.

System & Environment Requirements

Requirement Category	Specification / Tool
Low-Code Platform	Zoho Creator (Professional / Education Edition)
Pro-Code Platform	Zoho Catalyst (Serverless Microservices Platform)
Runtime Environment	Node.js (v18.x or above)
API Testing Tool	Postman / cURL
Scripting Languages	Deluge (Zoho Creator) & JavaScript / ES6 (Catalyst)
Browser & Connectivity	Modern Web Browser with Active Internet Connection

Architecture & Integration Flow

Flow Layer	Component	Functionality
Frontend / UI	Zoho Creator Form	Captures user request and inputs

Flow Layer	Component	Functionality
Orchestration	Deluge Script (Workflows)	Triggers REST API call via `invokeurl` task
Middleware / API	Catalyst API Gateway	Routes secured HTTP requests to serverless function
Pro-Code Backend	Catalyst Node.js Function	Executes complex computation, data validation, and logic
Response Path	JSON Payload	Returns processed calculation back to Creator form/record

Procedure

1. Create the Application

- Develop a primary application in Zoho Creator containing necessary forms, reports, pages, and basic workflows to manage regular operational workflows.

2. Set Up Zoho Catalyst

- Sign in to the Zoho Catalyst platform (`catalyst.zoho.com`) using developer credentials.
- Create a new project dedicated to housing backend microservices, serverless functions, and database components required for extending the low-code application.

3. Develop Backend Logic

- Select Node.js as the runtime environment.
- Write custom serverless functions to handle advanced tasks (such as complex matrix algorithms, large batch validation, or secure token cryptographic tasks) that exceed standard low-code Deluge capabilities.

```
// Catalyst Serverless Advanced Calculation Function (index.js)
const catalyst = require('zcatalyst-sdk-node');
```

module.exports = async (cronOrRequest, response) ⇒ {
try {
const reqBody = cronOrRequest.body;
const baseAmount = parseFloat(reqBody.baseAmount || 0);
const taxRate = parseFloat(reqBody.taxRate || 0.18);
const discountPercentage = parseFloat(reqBody.discount || 0);

```
None
    // Complex calculation logic
    const discountedTotal = baseAmount - (baseAmount * (discountPercentage /
100));
    const taxAmount = discountedTotal * taxRate;
    const grandTotal = discountedTotal + taxAmount;

    response.status(200).send({
        status: "success",
        data: {
            baseAmount: baseAmount,
            discountedTotal: discountedTotal,
            taxAmount: taxAmount,
            grandTotal: grandTotal,
            processedAt: new Date().toISOString()
        }
    });
} catch (error) {
    response.status(500).send({ status: "error", message: error.message });
}
```

};

```
None

### 4. Create Serverless Functions
* Package and initialize the Catalyst function locally or directly through the
Catalyst Console.
* Set up input validations, exception handling, and response structured
formatting.
```

5. Configure API Endpoints
* Define standard HTTP methods (POST/GET) and secure execution URLs for the serverless function.
* Note down the generated API Endpoint URL and authorization headers required for external communication.

6. Integrate with Zoho Creator
* Open Zoho Creator and establish a secure **Connection** or configure a Deluge `invokeurl` script triggered during form events (e.g., `On Validate` or `On User Input`).

```deluge
// Deluge Integration Script (Zoho Creator Form Event)
payload = Map();
payload.put("baseAmount", input.Service_Amount);
payload.put("discount", input.Discount_Rate);
payload.put("taxRate", 0.18);

headers = Map();
headers.put("Content-Type", "application/json");

// Calling Zoho Catalyst API Endpoint
response = invokeurl
[
    url:
"https://project-catalyst.zoho-catalyst.com/baas/v1/project/function/execute"
    type: POST
    parameters: payload.toString()
    headers: headers
    connection: "catalyst_oauth_connection"
];

if(response.get("status") == "success")
{
    dataMap = response.get("data");
    input.Tax_Amount = dataMap.get("taxAmount");
    input.Grand_Total = dataMap.get("grandTotal");
}
```

7. Implement Advanced Features

- Execute high-volume data computations, execute custom mathematical scoring, or integrate external third-party SDKs via the Catalyst backend layer without affecting Creator frontend response time.

8. Test the Integration

- Input sample records into the Zoho Creator form and verify that the trigger passes data seamlessly to the Catalyst backend.
- Validate that response payloads are accurately received, parsed, and rendered in Creator fields.

9. Deploy the Solution

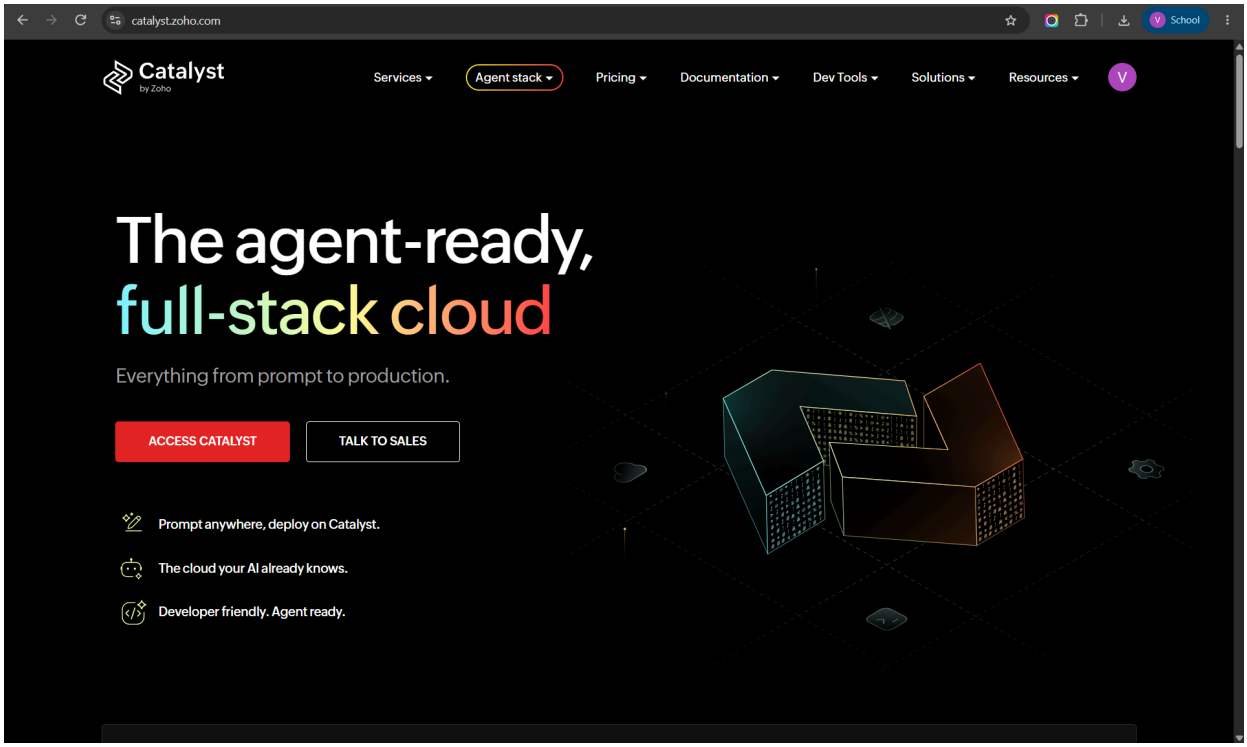
- Deploy the Catalyst functions to the Production environment (`catalyst deploy`).
- Configure required environment variables, CORS policies, and access tokens for live operation.

10. Verify the Application

- Perform end-to-end integration testing across various user roles to confirm seamless interaction between Zoho Creator and Zoho Catalyst while preserving all existing application workflows.

Result

The experiment successfully demonstrated how Zoho Catalyst can extend a Zoho Creator low-code application with custom backend logic, serverless functions, API integrations, and advanced processing capabilities. This approach seamlessly combines rapid low-code application development with pro-code flexibility to fulfill complex enterprise business requirements.


catalyst.zoho.com
School
Catalyst
by Zoho
Services
Agent stack
Pricing
Documentation
Dev Tools
Solutions
Resources
V
The agent-ready,
full-stack cloud
Everything from prompt to production.
ACCESS CATALYST
TALK TO SALES
Prompt anywhere, deploy on Catalyst.
The cloud your AI already knows.
Developer friendly. Agent ready.